

Two Pointers: A Framework for Problem Solving

1 What is the Two Pointer Technique?

The Two Pointer Technique is an optimization strategy where two indices (or pointers) traverse a data structure in a coordinated manner to reduce the time complexity, often from $O(n^2)$ to $O(n)$.

The key idea is **not** simply using two indices. Instead, each pointer has a well-defined responsibility while maintaining a certain property (called an **invariant**) throughout the algorithm.

Golden Rule:

Do not think:

“Where should I place two pointers?”

Instead think:

“**What should each pointer represent, and what property should always remain true?**”

2 Step-by-Step Framework

Whenever you encounter a problem, follow the steps below.

Step 1: Identify the Pattern

First classify the problem.

Pattern	Examples
Same Direction	Move Zeroes, Remove Duplicates
Opposite Direction	Two Sum II, Container With Most Water
Sliding Window	Longest Substring Without Repeating Characters
Fast & Slow	Linked List Cycle, Middle of Linked List
Partitioning	Dutch National Flag, Quicksort Partition

Ask yourself:

- Are both pointers moving in the same direction?
- Are they moving toward each other?
- Is one pointer chasing another?

Step 2: Define the Meaning of Each Pointer

Avoid naming them immediately as i and j .

Instead define what they represent.

Examples:

Move Zeroes

$left \rightarrow$ next position where a non-zero should be placed
 $right \rightarrow$ current element being examined

Remove Duplicates

$left \rightarrow$ last unique element
 $right \rightarrow$ current element

Sliding Window

$left \rightarrow$ start of current window
 $right \rightarrow$ end of current window

Never think of pointers as merely indices.

Step 3: Define the Invariant

This is the most important step.

An invariant is a condition that remains true after every iteration.

Ask yourself:

”What should always remain true?”

Examples:

Move Zeroes:

$[0 \dots left - 1]$

contains every non-zero element encountered so far.

Remove Duplicates:

$[0 \dots left]$

contains all unique elements.

Sliding Window:

The window

$[left, right]$

always satisfies the required condition.

The algorithm should never violate its invariant.

Step 4: Decide Which Pointer Explores

Usually one pointer explores while the other adjusts.

Ask:

- Which pointer always moves?
- Under what condition should the second pointer move?

Examples:

Move Zeroes

- right scans every element
- left moves only after placing a non-zero

Sliding Window

- right expands the window
- left shrinks the window when it becomes invalid

Step 5: Decide the Action

Every iteration usually performs one operation.

Examples include

- swap
- overwrite
- count
- expand
- shrink
- compute answer

Ask:

What happens when the required condition is satisfied?

Step 6: Update the Pointers

After performing the action, update the pointers.

General structure:

```
while (...)  
  
    inspect  
  
    perform action  
  
    move pointers
```

3 Worked Example: Move Zeroes

Problem:

Move every zero to the end while preserving the order of non-zero elements.

Pointer Meaning

$left$ = next position for a non-zero

$right$ = current element being scanned

Invariant

After processing index $right$,

$[0 \dots left - 1]$

contains all non-zero elements in their original order.

Action

If

$nums[right] \neq 0$

swap

$nums[left]$

and

$nums[right]$

then increment

$left$.

Always increment

$right$.

4 General Mental Checklist

Whenever solving a Two Pointer problem, ask these six questions:

1. What does each pointer represent?
2. What invariant must always hold?
3. Which pointer explores?
4. When should the second pointer move?
5. What action occurs when the condition is satisfied?
6. Does every pointer move at most n times?

If every pointer moves at most once through the data,

Time Complexity = $O(n)$

5 Common Two Pointer Patterns

Problem	Left Pointer	Right Pointer
Move Zeroes	Next write position	Scanner
Remove Duplicates	Last unique element	Scanner
Merge Arrays	Output position	Scanner
Sliding Window	Window start	Window end
Two Sum II	Smallest value	Largest value
Container With Water	Left boundary	Right boundary
Fast/Slow Linked List	Slow node	Fast node
Dutch National Flag	Boundary	Scanner

6 Key Insight

The success of a Two Pointer algorithm comes from maintaining a carefully chosen invariant.

Pointers are simply tools.

The invariant is the real algorithm.